From Code Archival to Knowledge Graph: Bridging Software Heritage, COAR Notify and Wikidata

Camillo Carlo Pellizzari di San Girolamo[1], Francesco Tosoni[2,*]

[1]Scuola Normale Superiore, p.zza dei Cavalieri 7, 56126 Pisa PI, Italy
[2]Sant'Anna School of Advanced Studies, L'EMbeDS, p.zza Martiri della Libertà 33, 56127 Pisa PI, Italy

Abstract

Software is a first-class scientific object, yet validated links between source code and the scholarly record remain largely absent from the Linked Open Data (LOD) cloud, isolating archived artefacts from semantic discovery. This paper presents an end-to-end reconciliation pipeline that harvests, validates, and models publication-to-repository pairs from sources where the link between a paper and its source code is explicit and editorially verified: the software-centric journals JOSS, SoftwareX, and IPOL, together with the reproducibility reports of the SIGMOD Availability and Reproducibility Initiative (ARI). This yields a curated corpus of 4,397 ⟨DOI, repository-URL⟩ pairs. We design two distinct application profiles grounded in Wikidata classes (one for scholarly articles, one for software instances) aligned with the schema.org and CodeMeta vocabularies. This architectural separation enables rule-based reconciliation at two granularities: lightweight, inline publication references or standalone, first-class Wikidata software nodes equipped with SWHIDs, Software Heritage's content-addressed identifiers. A read-only lookup against Wikidata shows that only 82 of the harvested repositories were already modelled there; human-reviewed batches have since created 4,182 new software items cross-linked to their articles. We further show that payloads of the emerging COAR Notify protocol, an external effort we do not develop, map natively onto our input format, so the same backend could later serve a live enrichment stream. Our core contribution is a pair of application profiles that turn Wikidata into a connector between the scholarly record and archived source code; we openly release all code, application profiles, and harvested datasets.

Keywords

Software Heritage, Wikidata, knowledge graph, entity reconciliation, COAR Notify, software citation, Linked Open Data

1. Introduction

Reproducibility and discoverability are foundational to science, yet the transition to software pipelines has eroded both: investigators fail to validate roughly 70% of published results, and about half cannot reproduce their own findings [1]. A primary cause is lost or unobtainable source code. As Karl Popper observed [2, §22], "Non-reproducible single occurrences are of no significance to science." Durable, unambiguous software identification is therefore a prerequisite for scientific trust, requiring code to survive and remain linked to the scholarly record.

Although software encodes modern scientific methodology, it is rarely cited as a formal, resolvable object [3, 4]. Two complementary infrastructures address this gap. Software Heritage (SWH) [5, 6, 4] archives public source code and assigns persistent, content-addressed identifiers (SWHIDs) via a Merkle directed acyclic graph (DAG) [7]. Meanwhile, Wikidata [8, 9, 10] (the central knowledge base of the Linked Open Data cloud) models software items alongside metadata such as DOIs and SWHIDs.

Each infrastructure solves half the problem: SWH guarantees preservation, while Wikidata provides a queryable semantic layer connecting software to papers, authors, and licences. Systematic bridging is missing. Connecting these graphs enriches discovery, enabling queries by functional properties (e.g.,

Wikidata'26: Wikidata Workshop at ISWC 2026
*Corresponding author.
camillo.pellizzaridisangirolamo@sns.it (C. C. Pellizzari di San Girolamo); Francesco.Tosoni@santannapisa.it (F. Tosoni)
https://www.sns.it/en/persona/camillo-carlo-pellizzari-di-san-girolamo (C. C. Pellizzari di San Girolamo);
https://www.santannapisa.it/en/francesco-tosoni (F. Tosoni)
0000-0003-2699-1693 (C. C. Pellizzari di San Girolamo); 0000-0001-8457-3866 (F. Tosoni)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

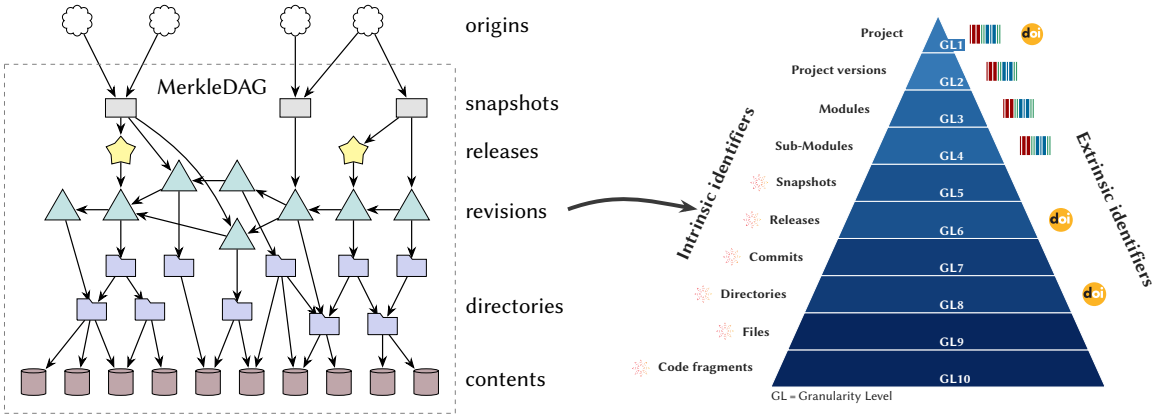


Figure 1: Granularity levels of the Software Heritage Merkle DAG [5] and corresponding identifier schemes [22]. *Intrinsic* SWHIDs address content-defined layers (from snapshots and releases down to code fragments) and are derived from the objects themselves. *Extrinsic* identifiers minted by external authorities attach wherever a human-meaningful unit is deposited and cited: typically a project, version or release, but also a directory when an archive mints a DOI for a deposited source tree. Property P6138 allows a project-level Wikidata item to reference its underlying intrinsic SWHID anchor.

language, licence, topic, or author) rather than solely by URL or hash, which also makes such data reusable by developers outside the knowledge-graph community [11, §5.7].

Building this bridge aligns with research-software policy: funders ask that software be treated as a first-class scholarly output and preserved in archives such as SWH [3], and research-software practitioners call for interconnected portals that integrate software into the scholarly record [12], alongside the scholarly graph that WikiCite has grown in Wikidata [13]. In France, the IPOL journal already deposits its code in SWH [14] and national prizes reward research software [12, 15].

The scale of this semantic gap is stark. As of July 2026, Wikidata's code-repository property (P1324) appears on 22,534 items, but only 133 include a DOI (P356), and only 65 carry both a SWHID (P6138) and a repository link. Although the ontological scaffolding exists, it remains sparsely populated. This paper designs and prototypes the reconciliation machinery required to fill it.

Beyond provenance, content-addressed links from papers to archived code let users navigate to runnable code, study software empirically at scale, and ground code search engines (e.g., MediaWiki Code2Code Search [16]) and code models (such as StarCoder2 [17]) in the scholarly record, complementing bibliographic graphs like DBLP [18].

Our contribution Our primary outcomes are fourfold: (i) we consolidate editorially verified software-journal and reproducibility-report sources into an open corpus of 4,397 curated ⟨DOI, repository⟩ pairs whose paper-to-code link is correct by construction; (ii) as our central contribution, we model software-citation links using separate application profiles for scholarly articles and software instances aligned with schema.org [19] and CodeMeta [20], turning Wikidata into a connector that reconciles SWH with the scholarly record at two granularities; (iii) we present a rule-based reconciliation pipeline that generates auditable, provenance-tagged candidate statements for human review; (iv) we show that a payload of the emerging COAR Notify protocol [21] maps directly onto our pipeline's input format, so the offline prototype and a future live notification stream could share a single processing backend.

2. Background

2.1. Software Heritage

Software Heritage (SWH) [5] was launched in 2016 to collect, preserve, and share all publicly available software source code [23, 6]. As of December 2025, it archives more than 27 billion unique source files from over 421 million projects, totalling roughly 2 petabytes [24].

SWH occupies a distinct place among code platforms. Forges such as GitHub or GitLab are commercial services that host active development, where projects can be renamed, moved, or deleted at will, while deposit repositories such as Zenodo mint a DOI for a snapshot that authors upload themselves. SWH is instead a non-profit archive, initiated by Inria in partnership with UNESCO and funded by public and private sponsors [23, 6]: it hosts no development, but proactively crawls forges and package managers and preserves their full history. Replicated to independent mirrors, any archived artefact can be retrieved, redistributed, and cited through its intrinsic identifier wherever it was first published, so the code behind a result stays verifiable without depending on a single provider.

The archive is structured as a five-level Merkle DAG [6, 7, 4] representing contents, directories, revisions, releases, and snapshots, rooted at origins [7]. Because the graph is content-addressed, identical files collapse into a single node shared across all projects, enabling global deduplication and a stable identity model. This design ensures resilience against hosting platform closures: when Gitorious and Google Code shut down, SWH retrieved and preserved full copies of their repositories [23, 4].[1]

Every archived object receives a SWHID (SoftWare Hash IDentifier), an intrinsic identifier derived directly from its content, rendering it independent of hosting platforms and immune to link rot [4, 25]. SWHIDs are codified under the ISO/IEC 18670:2025 standard [26] and supported via the Wikidata property P6138. Figure 1 contrasts the intrinsic SWHID with extrinsic identifiers, such as Wikidata Q-items and DOIs (including identifier systems like RRID [27]), which attach at coarser, human-meaningful levels, such as projects or releases. The archive also supports research on programming-language evolution [28], code language models trained on The Stack v2 [17], reproducible deployment [29], and cybersecurity [30], and the UNESCO Recommendation on Open Science [31] lists open source software and source code among the pillars of open scientific knowledge.

2.2. Wikidata

Wikidata is a collaborative, multilingual knowledge base serving as the central hub of the Linked Open Data cloud [8, 9, 10]. It is structured into subject-property-value triples, which can be qualified and equipped with references to track provenance. This suits software metadata, which evolves across versions, although curating such a graph remains challenging [18][11, §3.8].

Software items in Wikidata are modelled as instances of *software* (Q7397) and its subclasses, using established properties such as source code repository (P1324), DOI (P356), and SWHID (P6138).

This article-versus-software distinction is reflected in Wikidata's physical infrastructure. In 2025, the Wikimedia Foundation split the Wikidata Query Service (WDQS) into two SPARQL endpoints to overcome backend scaling limits [32]. One endpoint serves the scholarly graph, which is primarily WikiCite data [13] and accounts for over half of Wikidata's triples [32], while the main endpoint hosts software items (Q7397). As of July 2026, queries on WDQS that join a paper to its corresponding software must still span both graphs via internal federation, an architectural constraint we address in Section 4.4.

2.3. COAR Notify and the Reconciliation Trigger

The COAR Notify protocol defines machine-actionable notifications, expressed via Activity Streams 2.0, for scholarly communication workflows [21, 33]. The COAR community develops it; Cancellieri et al. [33] have blueprinted its application to software-citation discovery at scale as part of the SoFAIR project, but have not yet deployed it operationally. We neither develop this protocol nor operate its notification infrastructure: we treat it strictly as an external, prospective input channel and show only that its payloads align with what our pipeline already consumes.

Open Access (OA) repositories expose their deposits via OAI-PMH conforming to the RIOXX profile [34], an application profile that extends Dublin Core with structured fields for funder identifiers, licensing terms, and author ORCIDs. This common representation lets CORE harvest both the deposited

[1] See Software Heritage, *Software is fragile*: https://www.softwareheritage.org/mission/software-is-fragile/.

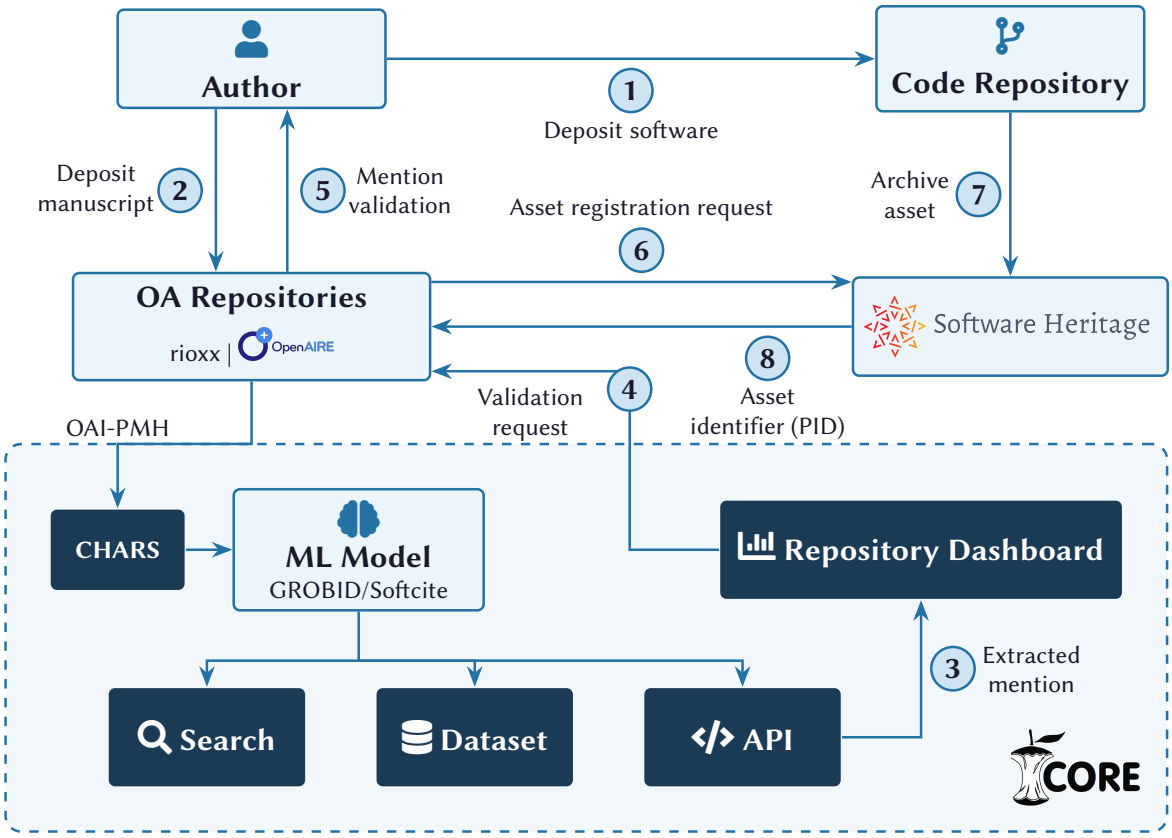


Figure 2: COAR Notify ecosystem for software-citation discovery and archival, after the SoFAIR workflow of Cancellieri et al. [33] (numbered arrows indicate workflow steps). Authors deposit code to a hosting platform and the corresponding manuscript to an OA repository (1–2); CORE harvests the deposit over OAI-PMH and extracts software mentions via its ML pipeline (3); the Repository Dashboard then routes a validation request to the repository (4), which forwards it to the author (5); once validated, the repository registers the asset with Software Heritage, which archives it and returns its persistent identifier (6–8).

PDFs and their provenance metadata: CHARS downloads the PDFs;[2] the GROBID suite [35, 36] parses each into structured XML (TEI) representations; Softcite isolates software mentions from that structure; and the Repository Dashboard sends a validation request back to the originating OA repository as a COAR Notify message, which the repository forwards to the author for approval (arrows 3 to 5 in Figure 2).

In our architecture, each COAR Notify `Announce` payload carries a paper's DOI alongside the repository URL of related software, mapping directly to the ⟨DOI, repo⟩ pairs consumed by our reconciliation pipeline (Section 5). These pairs are then reconciled against Wikidata and serialised as QuickStatements [37] to mint or enrich software items with repository links (P1324) and SWHIDs (P6138), at node or reference level depending on whether the paper presents or merely uses the software (Section 4.4), making COAR Notify a natural on-ramp for a future live enrichment stream.

3. Datasets: Original Sources and Derived Pairs

The systematic survey of entity linking with Wikidata by Scharpf et al. [8] evaluates 65 publications, 17 benchmark datasets, and 34 approaches. Crucially for our purposes, the only ones it describes as covering software mentions in scientific literature are Softcite [38] and SoMeSci [39]. Both are mention-level corpora: they annotate where a paper refers to third-party software so a single article can point to many tools, and they recover repository URLs from free text, which makes them broad in

[2]Component names follow the SoFAIR workflow documentation, https://sofairoa.github.io/documentation.

coverage but noisy. This scarcity and noise motivate building the corpus instead from sources where the paper-to-code link is explicit and editorially verified. Preprint full text is a complementary avenue: arXiv distributes the LaTeX sources of its papers, and corpora such as unarXive [40] convert them into over a million plain-text documents. Mining that text for availability statements (patterns such as "code is available at" followed by a URL) would raise recall at the cost of editorial verification; we leave that cross-check to future work.

We distinguish *original* datasets, the bibliographic sources we consume, from *generated* datasets, the ⟨DOI, repo⟩ pair tables our harvester derives from them (Table 1). In our sources, the editorial workflow ties each publication to a repository, so the link is correct by construction rather than inferred from prose.

3.1. Software-Journal Ground Truth

We gathered repository links for the three software journals using a tiered, API-first strategy. We queried the JOSS JSON catalogue directly (3,596 records covering 3,546 distinct works, with only one lacking a repository) and covered SoftwareX via Crossref API queries by ISSN (2,326 works). Because IPOL lacks a machine-readable endpoint, we fetched its landing pages once and cached them locally, then extracted the SWHID from the exposed BibTeX records (295 papers). After URL normalisation, SWHID resolution, and deduplication against the rest of the corpus, these venues contribute 3,545, 486, and 275 pairs, respectively, to the unified table (Table 1).

SoftwareX (Elsevier, ISSN 2352-7110) This journal requires a public code repository for every article, and Elsevier mirrors each accepted repository into a dedicated `ElsevierSoftwareX` GitHub organisation, nominally one mirror per paper. However, the article's Crossref metadata does not expose the repository link. We therefore harvested 2,326 SoftwareX DOIs from Crossref by ISSN, each carrying an Elsevier Publisher Item Identifier (PII), and independently enumerated the organisation's mirror repositories. We then recovered the paper-to-repository link through the PII embedded in each mirror's GitHub description, which serves as the join key back to the Crossref record, with a normalised-title match as a fallback.[3] For the `P1324` value, we record the mirror's upstream parent (the authors' original repository) rather than the Elsevier mirror itself; of the 665 mirrors matched to an article, only the 486 exposing such a parent enter the corpus. In contrast, we retain the mirror URL as the provenance reference for the mapping.

The Journal of Open Source Software (JOSS, ISSN 2475-9066) This venue represents an even stronger paradigm [41]: a JOSS paper briefly describes a software repository, where the repository URL is a mandatory submission field and peer review is conducted directly on the codebase. The mapping from a JOSS DOI to its repository is therefore one-to-one and trivially machine-extractable, making JOSS an ideal gold standard.[4]

The Image Processing On Line journal (IPOL, ISSN 2105-1232) The venue, hosted at https://www.ipol.im/, offers the most robust case. IPOL publishes image-processing algorithms alongside their reference implementations and systematically deposits their source code into Software Heritage [14]. Because an archived article embeds its assigned SWHID directly within its exported BibTeX record, it yields a ⟨DOI, SWHID⟩ pair where the archival anchor (`P6138`) is established by design. As the only

[3] This join trusts the PII that Elsevier records in each mirror's description. We found one mirror, `ElsevierSoftwareX/SOFTX_2020_261` (upstream `PhotonSTR-18`), whose description cites the PII of an unrelated article (*cashocs*, DOI `10.1016/j.softx.2020.100646`) instead of its own (DOI `10.1016/j.softx.2020.100640`), yielding a spurious pairing. It surfaced as a duplicate DOI and was excluded from the import. We found no further collisions of this kind in the venue.

[4] A few submissions nonetheless list the paper's own support repository (its last path segment being `paper` or `joss_paper`) rather than the software; four such rows are excluded at import, mirroring the SoftwareX case above. Where the recorded link resolves to a branch view or a paper subfolder, it is trimmed to the repository root (see Section 5).

source in our study that provides native SWHIDs, IPOL bypasses both full-text URL extraction and downstream repository-to-SWHID resolution, thereby yielding an ideal seed dataset for node-level reconciliation.[5]

The SIGMOD Availability and Reproducibility Initiative (ARI) To test generality beyond venues specialised in research software, we include the SIGMOD ARI, which has published individual reproducibility reports for badged papers since 2020; because these reports are purpose-written, they reliably name the code repository. We scraped the ARI landing page to enumerate all badged papers for 2020–2025 (295 total); of these, only 102 carry an individual report PDF (the remaining badged papers list no standalone report). The reports for 2020–2023 are served directly by the ARI site, while 24 of the 72 badged 2024 papers have reports on the ACM Digital Library; the 2025 cohort (107 papers) links collectively to the proceedings rather than to individual reports and was therefore excluded. We extracted repository URLs by combining regex matching over the PDF text with annotation-link extraction from the PDF metadata, then ranked them by occurrence count and by the presence of artefact-signal terms (*reproducib*, *artefact*) in the repository path. To audit this heuristic, a large language model (Claude), blind to the regex output, named the primary artefact repository from each report's text, and we manually inspected every disagreement with the automated pick. The review found the heuristic to be correct in 90 of 102 reports (88.2%). It improved the remaining twelve: nine corrected (chiefly URLs truncated by PDF line-wrapping or by an over-eager suffix-stripping bug, plus one case where the authors' origin repository replaced an ARI reviewer's clone) and three recovered (one repository wrongly discarded by an over-broad filter and two artefacts hosted on self-hosted GitLab instances outside the major-host allowlist). The validated pipeline yields 91 pairs from the 102 available reports (89.2%); the eleven misses reference no permanent repository, distributing their artefacts instead through cloud drives, anonymised links, or non-public archives.[6]

These venues thus state which repository a paper *is about*, rather than which tools it *uses*, forming the precision-oriented ground truth on which the rest of the paper builds.

3.2. Generated Datasets

Table 1 aggregates the per-venue harvest; once normalised and de-duplicated, it forms a unified table of 4,397 ⟨DOI, repo⟩ pairs.

Once compiled, every unique repository URL in the unified table is submitted to Software Heritage's bulk *Save Code Now* API. This mechanism preserves the referenced code and initiates an archival crawl, making each origin's SWHID resolvable so it can be asserted as property `P6138` on its corresponding Wikidata item. The pipeline itself only reads Wikidata; edits reach the live graph solely through human-reviewed batches (Section 5).

[5] Because IPOL exposes no repository, the SWHID is the item's sole archival anchor. Of the 275 harvested pairs, 98 carry neither a repository nor a resolvable SWHID and are excluded at import, since the resulting node would have no anchor at all; the remaining 177 are imported as software items bearing a title-derived label, an `instance of` statement, the SWHID (`P6138`), and the cross-link to their article, with no `P1324`. Three SWHIDs harvested from the landing-page BibTeX were malformed, concatenating several archived snapshots from a single origin; we repaired each to the single authoritative qualified SWHID resolved via the Software Heritage API. One of these, `10.5201/ipol.2011.cm_fds` (*Finite Difference Schemes for MCM and AMSS*) in fact bundles two distinct implementations archived under separate SWH origins; following the multi-repository pattern of Section 4.3, we model it as a single article with two software items. Conversely, the two DOIs of *Image Interpolation with Geometric Contour Stencils* (`g_igcs`, 2011 and 2012) share one SWHID and hence one software item. IPOL thus contributes 177 software items across 178 import rows.

[6] At import, an audit of the SIGMOD artefact repositories removed thirteen of the ninety-one harvested pairs. A shared reproducibility mono-repo (`damslab/reproducibility`) cited by five distinct Apache SystemDS tools was filtered out, since one repository cannot stand as the source repository of five separate software items; one further row carried no repository, SWHID, or derivable label; and seven rows were reproducibility-artefact repositories whose slug is a generic `sigmod`*NN*`/reproducibility` name and whose paper names no tool from which to derive a clean label, so importing them would have minted mislabelled, low-quality items. Where the artefact repository is a reproducibility wrapper but the tool name is recoverable, the label is taken from the paper's short title (a single clean token such as *Grafite* or *PimPam*) or set explicitly (*SQLSolver*, *DivExplorer*, *AU-DB*, *Clonos*, *Tuplex*). These exclusions take SIGMOD from 91 harvested pairs to 78 imported.

Table 1
Generated ⟨DOI, repo⟩ pair datasets by venue (successful pairs; in parentheses, the records each venue was harvested from), and their union. No pair is shared across venues, so de-duplication removes nothing at this stage.

Source	Access	Pairs
JOSS	JSON catalogue API	3,545 (of 3,596)
SoftwareX	Crossref (ISSN 2352-7110)	486 (of 2,326)
IPOL	landing-page BibTeX	275 (of 295)
SIGMOD (ARI)	reproducibility report PDFs (2020–2024)	91 (of 102 with reports)
Unified	de-duplicated union	**4,397**

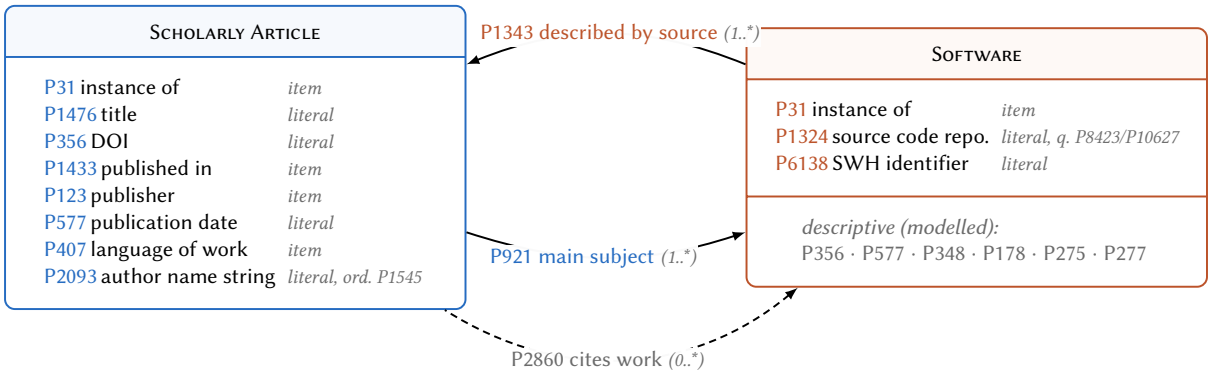


Figure 3: Cross-profile intersection between the Scholarly Article Profile (blue, grounded in Wikidata) and the Software Instance Profile (orange, grounded in Wikidata), showing the relational properties linking publications and code repositories. Solid arrows denote relations populated by this import; the dashed grey arrow denotes *cites work* (P2860), which is modelled but left for future work.

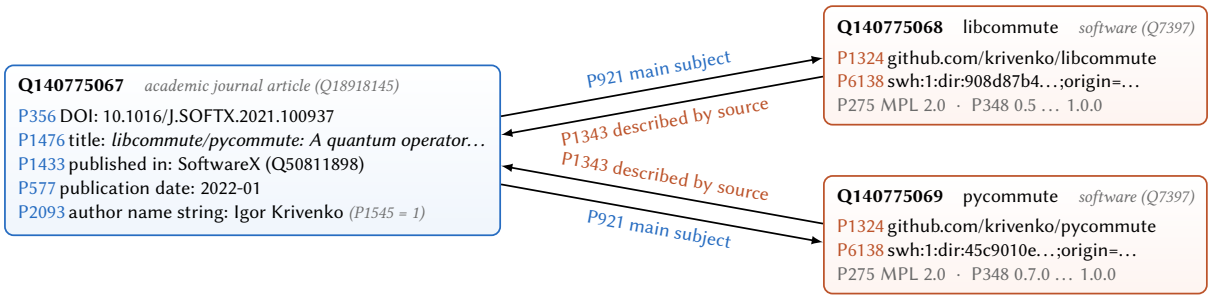


Figure 4: Running example: SoftwareX article `10.1016/j.softx.2021.100937` as imported into Wikidata, one article item and two software items joined by P921 and P1343 (statements abridged). A third-party bot (*Github-wiki-bot*) added the grey statements afterwards.

4. Two Application Profiles: Scholarly Articles and Software Instances

A software-citation link intersects two fundamentally distinct entities: a scholarly article and a software system. Rather than compressing them into a single record, we model each side with a dedicated application profile and connect them via explicit relations (Figure 3). This separation keeps the bibliographic identity of a paper distinct from the archival identity of the code it describes, allowing the reconciliation pipeline to determine the structural depth of each link dynamically.

Running example SoftwareX article `10.1016/j.softx.2021.100937` presents two related packages, `libcommute` and `pycommute`. Under our profiles it becomes three linked items (Figure 4): an *academic journal article* (Q140775067) carrying DOI, title, venue, and one *main subject* (P921) state-

Table 2
The scholarly-article profile and its crosswalk to schema.org. Upper block: properties populated by the current import. Lower block: modelled for the reference-level *uses* relationship (future work).

Wikidata property	schema.org	Role
Populated		
P31 instance of	`@type`	article class
P1476 title	`name` / `headline`	article title
P356 DOI	`identifier` (DOI)	citable identifier
P1433 published in	`isPartOf`	venue (journal / proc.)
P577 publication date	`datePublished`	publication date
P123 publisher	`publisher`	publisher (SIGMOD vols.)
P407 language of work	`inLanguage`	language
P2093 author name str.	`author`	authorship (ordinal P1545)
P921 main subject	`about`	subject (the software)
Modelled (future work)		
P2860 cites work	`citation`	software the paper *uses*

ment per package, and two *software* items (Q140775068, Q140775069[7]), each with its repository URL (P1324), SWHID (P6138), and a *described by source* (P1343) back-link. Within hours of their creation, a third-party bot added licence (P275) and version (P348) statements to both software items, showing how standalone software nodes accrue metadata (Section 4.4).

Both profiles reuse existing Wikidata classes and properties, a deliberate choice given that engineering new properties is a notorious bottleneck in collaborative knowledge graphs [11, §3.7]. They are aligned with established vocabularies: schema.org for articles, and the CodeMeta exchange schema [20] for software.

4.1. The Scholarly-Article Profile

A publication is modelled as a Wikidata item instantiating *academic journal article* (Q18918145), a subclass of the *scholarly article* (Q13442814) type applied extensively by the WikiCite initiative [13]; we prefer the narrower class for the journal venues (SoftwareX, JOSS, IPOL), reserving the sibling *conference paper* class (Q23927052) for the SIGMOD conference proceedings. SIGMOD spans both types: papers in the journal *Proceedings of the ACM on Management of Data* take the journal-article class and are *published in* Q130602410, as in the pre-existing WikiCite records, whereas papers in the 2020–2022 conference volumes take the conference-paper class and no *published in* statement, since those volumes have no Wikidata item and we mint no placeholder; for SIGMOD we therefore also record the *publisher* (P123, ACM) on new articles. The article model populates the bibliographic properties of Table 2, recording authors as name strings (P2093) with a series ordinal (P1545) rather than as person items. A relational property, *main subject* (P921), bridges each software paper to its software target; a companion property, *cites work* (P2860), models the reference-level *uses* relationship and is reserved for future work over software-citation datasets (Section 4.3).

4.2. The Software-Instance Profile

A software system is represented as an instance of *software* (Q7397), or a specific subclass like *free software* (Q341) when supported by evidence. Its properties serve two distinct roles. *Identifying* properties establish cross-archive identity and serve as the primary join keys for reconciliation; the current import populates the source code repository (P1324, qualified by its version control system P8423 and web interface software P10627) and the SWHID (P6138), together with the back-link *described by source* (P1343) to the parent paper. *Descriptive* properties, which the model provides to

[7] Query: https://w.wiki/UjdF.

Table 3
The software-instance profile and its crosswalk to CodeMeta 3.0. Upper block: identity and cross-link properties populated by the current import (P1324 qualified by version control system P8423 and web interface software P10627). Lower block: descriptive properties the model provides, populated as evidence accrues. *Voc.* records the namespace each term resolves to in the CodeMeta context: sdo for schema: (schema.org), cm for CodeMeta's own codemeta:; @type is a JSON-LD keyword.

Wikidata property	CodeMeta term	Voc.	Role
Populated (identity + link)			
P31 instance of	@type	—	software class
P1324 source code repo.	codeRepository	sdo	repository URL (identity)
P6138 SWH identifier	identifier (SWHID)	sdo	archival anchor (identity)
P1343 described by source	referencePublication	cm	back-link to describing paper
Modelled (descriptive enrichment)			
P356 DOI	identifier (DOI)	sdo	software's own deposit DOI
P577 publication date	datePublished	sdo	release / version date
P348 software version id.	softwareVersion	sdo	version label
P178 developer	author	sdo	authorship
P275 copyright license	license	sdo	license
P277 programmed in	programmingLanguage	sdo	implementation language

enrich an item once its identity is resolved, are populated as such evidence accrues: an archival deposit DOI (P356) when an artefact is registered on a platform like Zenodo, release date (P577), software version (P348), developer (P178), copyright license (P275), and programming language (P277). Table 3 illustrates this schema and its crosswalk to CodeMeta 3.0, whose terms come from schema.org except referencePublication, which is CodeMeta's own.

This descriptive schema refines the repository-centric model Turki et al. [42] used to track GitHub repositories for Wikimedia tooling. Whereas they harvested transient platform metrics (such as star counts, forks, and last-push dates) across a single forge, our model foregrounds three descriptive properties core to citable identity, to be populated as evidence accrues: programming language (P277), license (P275), and developer (P178). We deliberately store both the repository URL (P1324) and the SWHID (P6138), since the two play complementary roles. The URL points to where development currently happens and may change when a project migrates or is renamed. In contrast, the SWHID is a permanent, content-addressed snapshot minted uniformly across forges, decoupling identity from any hosting platform and resisting link rot.

4.3. Linking the Two Profiles

The profiles intersect via directed relationships rather than shared identifier properties. From the article side, *main subject* (P921) indicates that a dedicated software paper is structurally *about* a software item, whereas *cites work* (P2860) models an ordinary citation where a paper simply *uses* a tool, a case we populate only from the software-mention datasets, left to future work. Conversely, the software item uses *described by source* (P1343) as an inverse back-link to its primary paper. This cross-profile bridge (Figure 3) is not strictly one-to-one, since a paper presenting several packages carries one P921 per software item (Figure 4).

This also resolves a common confusion about DOIs: on an article item, P356 identifies the publication, whereas on a software item it identifies an independent archival deposit (such as a Zenodo release). The two never coincide, avoiding the conflation common in single-record metadata schemas.

4.4. Two Levels of Reconciliation Granularity

Reconciling an SWH object with Wikidata can proceed at two granularities. At the coarser *reference level*, no dedicated software item is minted: the article item is retrieved or created via its DOI, and the

SWHID or repository URL is attached directly as a statement reference. This keeps archived source code discoverable from the bibliographic graph without full entity resolution, pointing to any layer of the Merkle graph down to individual files or revisions (Figure 1). It degrades gracefully when metadata is sparse, making it ideal for the long tail of incidental software mentions.

At the finer *node level*, software is promoted to a first-class Wikidata item (Q7397) carrying its own identity properties (P1324, P6138) and descriptive statements (P348, P275, P277). The item is linked to the publication via P921 on the article (reserving P2860 for reference-level usage) and P1343 on the software. This tier enables rich semantic queries (e.g., listing all papers citing a tool or mapping software licences across a discipline) but requires strict disambiguation to prevent duplicate nodes. Following the 2025 query-service split that isolates articles and software across separate SPARQL endpoints [32], our cross-entity properties (P921, P1343) provide explicit keys for efficient federated joins on WDQS, while QLever offers an alternative endpoint.[8]

Node level is our default for software that is the explicit subject of a paper, as in our venues. Articles and code routinely carry different licences (e.g., CC BY for a paper, MIT for its code), so P275 needs a dedicated software node, and standalone nodes accrue versions and statements without later disentangling reference links (Figure 4). The reference level is reserved for incidental mentions.

4.5. Mapping Software Heritage Objects

Node-level identity mirrors the Merkle graph topology. Each SWH *origin*, defined by its upstream repository URL, maps to a single software item, with the URL serving as the P1324 value and the primary reconciliation key. Snapshots and revisions beneath this origin capture version-level identity: the SWHID of a tagged release or snapshot is populated as a P6138 value, which the model further qualifies with a version string (P348) and release date (P577) where available. Because citations generally target the software concept rather than an isolated commit, we anchor node identity at the origin level, recording fine-grained SWHIDs as qualified, version-scoped statements to control concept-versus-version ambiguity.

Bibliographic and identifier statements carry a structured reference block capturing the source corpus, evidence URL or DOI, and extraction date.

5. The Reconciliation Pipeline

The pipeline is the core component of an open-source library released alongside our harvested datasets for full reproducibility.[9] It executes five sequential stages (harvesting, bulk archiving, lookup, statement generation, and a reviewed two-pass import), following knowledge-engineering best practices by starting from explicit use cases and enforcing strict provenance for every claim [11, §5.8].

Harvesting normalises heterogeneous metadata into a uniform table of ⟨DOI, repo-URL⟩ pairs. A tiered API strategy across three software journals and SIGMOD ARI reports (Section 3.1) yields 4,397 pairs (Table 1).

Bulk archiving normalises harvested URLs, filters out non-code hosts, and discards existing SWH links, retaining only origins on a code-hosting allowlist (CODE_HOST). This filters out generic homepages, docs, and direct downloads. The remaining origins are submitted to SWH's bulk *Save Code Now* API, yielding 4,086 unique repositories to archive from the initial 4,397 pairs. *SWHID retrieval* then queries visit histories, extracts the crawl closest to publication, and resolves 4,244 pairs to a qualified P6138 SWHID value (with the remainder retaining repository URLs).

Lookup decides, for each pair, whether to create or enrich. In our experience, the only reliable identity keys are exact identifiers: an article is matched by its DOI (P356, case-insensitively) and a software item by its normalised repository URL (P1324), with the SWHID (P6138) as a secondary key (Section 6). Name-based matching is never used to merge items, since software labels are short and heavily reused

[8] https://qlever.dev/wikidata/

[9] Code, data, and import batch reports: https://github.com/ftosoni/swh-wd-reconciliation.

Table 4
Lookup outcome against the live Wikidata graph (read-only, 29 July 2026). Software nodes are matched by repository URL (P1324); articles by DOI (P356).

		Software		Article	
Venue	Pairs	exist.	new	exist.	new
SoftwareX	486	6	480	210	276
JOSS	3,545	76	3,469	1,691	1,854
IPOL	275	0	275	31	244
SIGMOD	91	0	91	35	56
Total	**4,397**	**82**	**4,315**	**1,967**	**2,430**

(e.g., two unrelated projects named `umami`). We create missing nodes, while enriching existing ones rather than duplicating them. Pairs that would collapse distinct entities onto one node, such as a repository shared by several tools or a publisher identifier collision (Section 3.1), are excluded after manual review rather than merged, preventing item conflation and duplication [43].

Statement generation turns the validated pairs into CSV tables, imported through OpenRefine or converted into QuickStatements commands, with target QIDs (or new-item placeholders), properties, values, and provenance blocks. Article titles are stripped of JATS/MathML markup and HTML entities, with whitespace collapsed to create clean labels and P1476 values. Repository URLs serving as P1324 values are trimmed of sub-paths (e.g., `/tree/`, `/src/`, or `/issues`) back to the repository root. To keep label and description jointly unique, software items with ambiguous names (such as the two `umami` projects) append the repository slug to their description. Granularity choices follow the policy in Section 4.4.

Import runs in two passes, because a batch cannot use an item it creates as the value of another statement. The first pass creates the missing software and article items (via OpenRefine or QuickStatements 3.0); the second, fed with the QIDs listed in the first pass's run report, completes the P921/P1343 cross-links and adds the author strings. The authors inspected and ran every batch, creating 4,182 software and 2,326 article items. Our code repository lists every batch (also browsable on EditGroups) and includes the QuickStatements run reports; once imported, the statements become part of Wikidata's public RDF graph, queryable live through its SPARQL endpoints and redistributable through its RDF dumps.

6. Evaluation

We evaluate the pipeline's *lookup* stage, which, for each harvested node, determines whether an equivalent item already exists in Wikidata by querying the live graph directly. This both quantifies the enrichment opportunity and prevents duplicate nodes from being created before any edit is staged.

A read-only query against the live Wikidata graph (retrieved 29 July 2026) resolves each of the 4,397 ⟨article, software⟩ pairs from the four venues against existing items.[10] A software node is matched by its source code repository URL (P1324), with the SWHID (P6138) as a secondary key; an article node is matched by its DOI (P356). Repository URLs are canonicalised on both sides before comparison (lower-casing the whole URL and stripping the scheme, a leading `www.`, trailing slashes, and `.git` suffixes). Table 4 breaks the outcome down by venue: only 82 of the 4,397 software repositories (1.9%) already have a Wikidata item, so 4,315 are candidates for new software nodes, whereas 1,967 (44.7%) of the articles already exist through WikiCite. These counts span the entire harvested corpus, before the per-venue editorial refinements in Section 3.1, so they measure the enrichment opportunity rather than any single import batch.

[10]The exact read-only SPARQL SELECTs (indexing P1324/P6138 for software on the main endpoint and matching P356 for articles on the scholarly subgraph) are part of the released code (`precheck_wikidata.py`); none writes to Wikidata.

Because venue policies fix the coupling between a DOI and its repository, the corpus also gives an objective precision baseline: a mapping that pairs a venue DOI with a different repository is incorrect. Computing structural precision metrics on the full corpus is ongoing work.

A primary limitation of the current prototype is that repository matching relies on URL equality after canonicalisation rather than graph traversal within SWH. Canonicalisation absorbs superficial variants, but the match still produces false negatives when projects migrate between hosting platforms (e.g., from Bitbucket to GitHub) without corresponding updates to their Wikidata `P1324` values. Anchoring identity directly on the SWHID, which remains invariant across code migrations, is our primary planned technical improvement.

Lessons learned Most defects arose during extraction rather than modelling, and manual audits caught them all before upload. They took three forms: publisher metadata pointing to a sibling record (the SoftwareX identifier collision); links corrupted in transit, such as URLs broken by PDF line-wrapping in SIGMOD reports or concatenated SWHIDs in IPOL exports; and repositories that break the one-repository-per-item assumption, being shared by several tools or holding only the paper (Section 3.1). We therefore recommend that venues publish the software link (repository URL or SWHID) as structured metadata, as JOSS and IPOL already do through their catalogue and BibTeX exports and ideally in the DOI record itself, rather than as free text in a PDF or a publisher-side mirror.

7. Conclusion

We presented an end-to-end pipeline that reconciles software between Software Heritage and Wikidata over 4,397 editorially verified ⟨DOI, repository⟩ pairs, of which only 82 repositories were already in Wikidata. Its contributions are threefold: dual application profiles that map SWH concepts onto Wikidata while keeping publications and software distinct at the reference or node level; an input format that natively accepts COAR Notify payloads; and reviewable, provenance-tagged upload batches that prevent unvetted writes to the live graph.

Immediate next steps involve measuring stage-by-stage precision, connecting the backend to a live COAR Notify stream, and extending harvesting to venues that mandate artefact links, such as the Resource tracks of ISWC and ESWC. Finally, adding multilingual labels and descriptions remains essential to maximise Wikidata's utility as a cross-lingual hub.

Acknowledgments

This study was funded by Wikimedia Italia through its micro-grant programme (https://w.wiki/UHgf). Francesco Tosoni was funded by the Alfred P. Sloan Foundation with the grant #G-2025-25193 (sloan.org). The authors thank the anonymous reviewers for their constructive comments and Morane Gruenpeter for helpful discussions.
Figure 2 adapts the SoFAIR workflow of Cancellieri et al. [33] (CC BY 4.0) and uses Font Awesome Free icons (CC BY 4.0). All logos are trademarks of their respective owners.

Declaration on Generative AI

During the preparation of this work, the authors used Claude (Anthropic) to draft content, paraphrase, reword, and generate images, and Grammarly to improve writing style. In more detail, the authors used Claude (Anthropic) to rewrite their data import logs and reconciliation notes in discursive form to produce a preliminary draft of the corresponding sections, shorten and reword text, and write the TikZ code for the figures. The released code was also written with assistance from Claude (Anthropic) and reviewed through a manual code-review process. After using these tools, the authors reviewed and edited the content as needed, checked every citation against the cited works, and take full responsibility for the publication's content.

References

[1] S. Greengard, Is there a way to solve the reproducibility problem?, Communications of the ACM (2026). URL: https://cacm.acm.org/news/is-there-a-way-to-solve-the-reproducibility-problem, news.

[2] K. Popper, The Logic of Scientific Discovery, Hutchinson, London, 1959. Routledge Classics reprint, London, 2002, ISBN 978-0-415-27844-7.

[3] C. Strasser, K. L. Hertweck, J. Greenberg, D. Taraborelli, E. Vu, Ten simple rules for funding scientific open source software, PLoS Comput. Biol. 18 (2022) e1010627. doi:10.1371/JOURNAL.PCBI.1010627.

[4] R. Di Cosmo, M. Gruenpeter, S. Zacchiroli, Referencing source code artifacts: A separate concern in software citation, Computing in Science & Engineering 22 (2020) 33–43. doi:10.1109/MCSE.2019.2963148.

[5] R. Di Cosmo, S. Zacchiroli, The Software Heritage open science ecosystem, in: Software Ecosystems: Tooling and Analytics, Springer International Publishing, Cham, 2023, pp. 33–61. URL: https://doi.org/10.1007/978-3-031-36060-2_2. doi:10.1007/978-3-031-36060-2_2.

[6] J.-F. Abramatic, R. Di Cosmo, S. Zacchiroli, Building the universal archive of source code, Commun. ACM 61 (2018) 29–31. doi:10.1145/3183558.

[7] A. Pietri, D. Spinellis, S. Zacchiroli, The Software Heritage graph dataset: Public software development under one roof, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 138–142. doi:10.1109/MSR.2019.00030.

[8] P. Scharpf, C. Breitinger, A. Spitz, N. Meuschke, A. Greiner-Petter, M. Schubotz, B. Gipp, Entity linking with Wikidata: A systematic literature review, ACM Comput. Surv. 58 (2026) 227. doi:10.1145/3795134.

[9] D. Vrandečić, M. Krötzsch, Wikidata: a free collaborative knowledgebase, Commun. ACM 57 (2014) 78–85. doi:10.1145/2629489.

[10] D. Vrandečić, L. Pintscher, M. Krötzsch, Wikidata: The making of, in: Companion Proceedings of the ACM Web Conference 2023, WWW '23 Companion, Association for Computing Machinery, New York, NY, USA, 2023, p. 615–624. doi:10.1145/3543873.3585579.

[11] P. Groth, E. Simperl, M. van Erp, D. Vrandečić, Knowledge Graphs and their Role in the Knowledge Engineering of the 21st Century (Dagstuhl Seminar 22372), Dagstuhl Reports 12 (2023) 60–120. URL: https://drops.dagstuhl.de/entities/document/10.4230/DagRep.12.9.60. doi:10.4230/DagRep.12.9.60.

[12] R. Di Cosmo, S. Granger, K. Hinsen, N. Jullien, D. Le Berre, V. Louvet, C. Maumet, C. Maurice, R. Monat, N. P. Rougier, Stop treating code like an afterthought: record, share and value it, Nature 646 (2025) 284–286. doi:10.1038/d41586-025-03196-0.

[13] D. Taraborelli, J. Dugan, L. Pintscher, D. Mietchen, C. Neylon, WikiCite 2016 Report, Technical Report, Wikimedia Foundation, 2016. URL: https://figshare.com/articles/journal_contribution/WikiCite_2016_Report/4042530. doi:10.6084/m9.figshare.4042530.v2.

[14] Software Heritage, IPOL archives research software in Software Heritage, 2020. URL: https://www.softwareheritage.org/2020/06/11/ipol-and-swh, accessed 2026-06-10.

[15] R. Di Cosmo, S. Granger, K. Hinsen, N. Jullien, D. Le Berre, V. Louvet, C. Maumet, C. Maurice, R. Monat, N. P. Rougier, CODE beyond FAIR: a roadmap for reusable research software, Scientific Data 13 (2026) 514. doi:10.1038/s41597-026-06705-6.

[16] F. Tosoni, MediaWiki Code2Code Search: Neural retrieval for the semantic discovery of open-source software entities, 2026. URL: https://arxiv.org/abs/2607.26766. arXiv:2607.26766.

[17] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, T. Liu, M. Tian, D. Kocetkov, A. Zucker, Y. Belkada, Z. Wang, Q. Liu, D. Abulkhanov, I. Paul, Z. Li, W.-D. Li, M. Risdal, J. Li, J. Zhu, T. Y. Zhuo, E. Zheltonozhskii, N. O. O. Dade, W. Yu, L. Krauß, N. Jain, Y. Su, X. He, M. Dey, E. Abati, Y. Chai, N. Muennighoff, X. Tang, M. Oblokulov, C. Akiki, M. Marone, C. Mou, M. Mishra, A. Gu, B. Hui, T. Dao, A. Zebaze, O. Dehaene, N. Patry, C. Xu, J. McAuley, H. Hu, T. Scholak, S. Paquet, J. Robinson, C. J. Anderson, N. Chapados, M. Patwary, N. Tajbakhsh,

Y. Jernite, C. M. Ferrandis, L. Zhang, S. Hughes, T. Wolf, A. Guha, L. von Werra, H. de Vries, StarCoder 2 and The Stack v2: The next generation, 2024. URL: https://arxiv.org/abs/2402.19173. arXiv:2402.19173.

[18] M. Ley, DBLP: some lessons learned, Proc. VLDB Endow. 2 (2009) 1493–1500. doi:10.14778/1687553.1687577.

[19] A. Iliadis, A. Acker, W. Stevens, S. B. Kavakli, One schema to rule them all: How Schema.org models the world of search, Journal of the Association for Information Science and Technology 76 (2025) 460–523. doi:10.1002/asi.24744.

[20] M. B. Jones, C. Boettiger, A. C. Mayes, A. Smith, M. Gruenpeter, V. Lorentz, T. Morrell, D. Garijo, P. Slaughter, K. Niemeyer, Y. Gil, M. Fenner, K. Nowak, M. Hahnel, L. Coy, A. Allen, M. Crosas, A. Sands, N. C. Hong, P. Cruse, D. S. Katz, C. Goble, B. Mecum, A. Gonzalez-Beltran, N. Ross, CodeMeta: an exchange schema for software metadata. Version 3.0, https://w3id.org/codemeta/3.0, 2023. Also available at https://codemeta.github.io.

[21] COAR Notify Initiative, COAR Notify — protocol specification and documentation, 2026. URL: https://coar-notify.net.

[22] M. Gruenpeter, Digital sovereignty in the age of software forges: Why preserving source code matters, Keynote presentation, 6th Conference for Research Software Engineering in Germany (deRSE26), Stuttgart, Germany, 2026. URL: https://doi.org/10.5281/zenodo.18848442. doi:10.5281/zenodo.18848442.

[23] R. Di Cosmo, S. Zacchiroli, Software Heritage: Why and How to Preserve Software Source Code, in: iPRES 2017 - 14th International Conference on Digital Preservation, Kyoto, Japan, 2017, pp. 1–10. URL: https://hal.science/hal-01590958.

[24] Software Heritage, Software Heritage Activity Report: 2025, Technical Report, Inria Foundation, Paris, France, 2026. URL: https://www.softwareheritage.org/2026/01/16/software-heritage-activity-report-2025/.

[25] R. Di Cosmo, Archiving and referencing source code with Software Heritage, in: A. M. Bigatti, J. Carette, J. H. Davenport, M. Joswig, T. de Wolff (Eds.), Mathematical Software – ICMS 2020, Springer International Publishing, Cham, 2020, pp. 362–373. doi:10.1007/978-3-030-52200-1_36.

[26] ISO/IEC, Information technology — SoftWare Hash IDentifier (SWHID) Specification V1.2, Technical Report ISO/IEC 18670:2025, International Organization for Standardization and International Electrotechnical Commission, 2025. URL: https://www.iso.org/standard/89985.html.

[27] A. Bandrowski, M. Brush, J. S. Grethe, M. A. Haendel, D. N. Kennedy, S. Hill, P. R. Hof, M. E. Martone, M. Pols, S. C. Tan, N. Washington, E. Zudilova-Seinstra, N. Vasilevsky, RINL Resource Identification Initiative, The Resource Identification Initiative: a cultural shift in publishing, Brain and Behavior 6 (2016) e00417. doi:10.1002/brb3.417.

[28] A. Desmazières, R. Di Cosmo, V. Lorentz, 50 years of programming language evolution through the Software Heritage looking glass, in: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), 2025, pp. 372–383. doi:10.1109/MSR66628.2025.00068.

[29] L. Courtès, T. Sample, S. Zacchiroli, S. Tournier, Source code archiving to the rescue of reproducible deployment, in: Proceedings of the 2nd ACM Conference on Reproducibility and Replicability, ACM REP '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 36–45. doi:10.1145/3641525.3663622.

[30] O. Barais, R. Di Cosmo, L. Mé, S. Zacchiroli, O. Zendra, Software Identification for Cybersecurity: Survey and Recommendations for Regulators, Technical Report, Software Heritage Security project (SWHSec), 2025. URL: https://hal.science/hal-05009757.

[31] UNESCO, UNESCO Recommendation on Open Science, Technical Report SC-PCB-SPP/2021/OS/UROS, United Nations Educational, Scientific and Cultural Organization, Paris, France, 2021. doi:10.54677/MNMH8546, adopted at the 41st session of the General Conference.

[32] T. Lubiana, L. Rasberry, D. Mietchen, The Wikidata Query Service split and its impact on the scholarly graph, in: D. Chaves-Fraga, I. Heibi, D. Garijo, D. Collarana, A. A. Salatino, S. Vahdati

(Eds.), Joint Proceedings of Posters, Demos, Workshops, and Tutorials of the 21st International Conference on Semantic Systems co-located with 21st International Conference on Semantic Systems (SEMANTiCS 2025), Vienna, Austria, September 3-5, 2025, volume 4064 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025. URL: https://ceur-ws.org/Vol-4064/PD-paper3.pdf.

[33] M. Cancellieri, M. Docekal, D. Pride, M. Gruenpeter, D. Douard, P. Knoth, Interoperable verification and dissemination of software assets in repositories using COAR Notify, 2025. URL: https://arxiv.org/abs/2508.02335. arXiv:2508.02335.

[34] Rioxx Governance Group, Rioxx: The research outputs metadata schema – Version 3.0, 2023. URL: https://www.rioxx.net/profiles/v3-0-final/.

[35] P. Lopez, GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications, in: M. Agosti, J. Borbinha, S. Kapidakis, C. Papatheodorou, G. Tsakonas (Eds.), Research and Advanced Technology for Digital Libraries, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 473–474. doi:10.1007/978-3-642-04346-8_62.

[36] M. Grennan, J. Beel, Synthetic vs. real reference strings for citation parsing, and the importance of re-training and out-of-sample data for meaningful evaluations: Experiments with GROBID, GIANT and CORA, in: P. Knoth, C. Stahl, B. Gyawali, D. Pride, S. N. Kunnath, D. Herrmannova (Eds.), Proceedings of the 8th International Workshop on Mining Scientific Publications, Association for Computational Linguistics, Wuhan, China, 2020, pp. 27–35. URL: https://aclanthology.org/2020.wosp-1.4/.

[37] Wikimedia Foundation, Help:QuickStatements – A tool for batch editing on Wikidata, Wikidata Help Page, 2026. URL: https://www.wikidata.org/wiki/Help:QuickStatements, tool: https://quickstatements.toolforge.org/. Accessed: 2026-06-17.

[38] C. Du, J. Cohoon, P. Lopez, J. Howison, Softcite dataset: A dataset of software mentions in biomedical and economic research publications, Journal of the Association for Information Science and Technology 72 (2021) 870–884. doi:10.1002/asi.24454.

[39] D. Schindler, F. Bensmann, S. Dietze, F. Krüger, SoMeSci- a 5 star open data gold standard knowledge graph of software mentions in scientific articles, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, CIKM '21, Association for Computing Machinery, New York, NY, USA, 2021, p. 4574–4583. doi:10.1145/3459637.3482017.

[40] T. Saier, M. Färber, unarXive: a large scholarly data set with publications' full-text, annotated in-text citations, and links to metadata, Scientometrics 125 (2020) 3085–3108. URL: https://doi.org/10.1007/s11192-020-03382-z. doi:10.1007/s11192-020-03382-z.

[41] A. M. Smith, K. E. Niemeyer, D. S. Katz, L. A. Barba, G. Githinji, M. Gymrek, K. D. Huff, C. R. Madan, A. C. Mayes, K. M. Moerman, P. Prins, K. Ram, A. Rokem, T. K. Teal, R. V. Guimera, J. VanderPlas, Journal of Open Source Software (JOSS): design and first-year review, PeerJ Comput. Sci. 4 (2018) e147. doi:10.7717/PEERJ-CS.147.

[42] H. Turki, M. A. H. Taieb, M. B. Aouicha, L. Rasberry, D. Mietchen, Preregistration: Comparing the use of Wikidata and Wikipedia by open-source software programmers on GitHub repositories, in: L. Kaffee, S. Razniewski, K. Alghamdi, H. Arnaout (Eds.), Proceedings of the Wikidata Workshop 2023 co-located with 22nd International Semantic Web Conference (ISWC 2023), Athens, Greece, November 13, 2023, volume 3640 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023. URL: https://ceur-ws.org/Vol-3640/paper10.pdf.

[43] C. C. Pellizzari di San Girolamo, Conflations and duplications in Wikidata items: Causes, detection, solutions, and issues, in: L. Kaffee, S. Razniewski, K. Alghamdi, H. Arnaout (Eds.), Proceedings of the Wikidata Workshop 2023 co-located with 22nd International Semantic Web Conference (ISWC 2023), Athens, Greece, November 13, 2023, volume 3640 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023. URL: https://ceur-ws.org/Vol-3640/paper4.pdf.